Database Design and Normalization Report

Introduction

This report outlines the conceptual design, normalization analysis, and technical schema layout for a relational database system. The objective is to ensure data integrity, optimize storage efficiency, and provide a clear structural reference for implementation. The following sections detail the classification of attributes, the proof of compliance with normalization standards, and a comprehensive data dictionary for the core entities.

Task 3 (a): System Conceptual Design and Attribute Classification

Before proceeding to the physical implementation of a database, it is essential to classify data conceptually. This process ensures that attributes are handled according to their inherent properties, facilitating optimal storage and retrieval. The attributes in this system are categorized into four primary relational database classifications:

1. Simple (Atomic) Attributes

Simple attributes consist of single, indivisible data values that cannot be logically decomposed further. These are the fundamental building blocks of a record.

- **Examples:** The `gender` field in the `STUDENTS` table, `total_modules` in the `COURSES` table, and `hours_per_wk` in the `SUBJECTS` table.

2. Composite Attributes

Composite attributes represent single conceptual entities that are composed of multiple sub-components. To achieve higher granularity and searchability in a relational database, these are typically “flattened” into individual columns.

- **Resolution:** In a Third Normal Form (3NF) schema, composite attributes are broken apart. For instance, a student’s **Full Name** is stored as discrete `first_name` and `last_name` columns within the `STUDENTS` table.

3. Single-Valued Attributes

Single-valued attributes are those permitted to hold only one specific value per record. They represent unique characteristics of an entity instance.

- **Examples:** The `date_of_birth` for a specific student or the `department_id`, which serves as a unique primary key identifier for a department.

4. Multi-Valued Attributes

Multi-valued attributes are conceptual fields that may contain multiple instances of data for a single entity. Because relational database engines generally forbid storing lists or arrays in a single cell, these must be resolved through structural changes.

- **Resolution:** These are managed by creating independent intersection or junction tables linked via Foreign Keys.
- **Examples:** A student's participation in multiple clubs is managed through the `CLUB_MEMBERS` intersection table. Similarly, a lecturer's workload across multiple classes is resolved using the `WORKLOAD_ALLOC` junction table.

Task 3 ©: Third Normal Form (3NF) Compliance Analysis

The proposed schema has been designed to comply with Third Normal Form (3NF) guidelines. This level of normalization is critical for preventing data anomalies (insertion, update, and deletion) and optimizing query performance.

Normal Form	Requirement	Compliance Proof
1NF	Eliminates repeating groups; every cell stores one atomic value.	The schema implements unique auto-incrementing integer Primary Keys (e.g., `student_id`). All repeating structural groups are eliminated.
2NF	Must be in 1NF; removes partial dependencies (non-keys must depend on the full PK).	For tables with composite keys, such as `WORKLOAD_ALLOC`, all secondary attributes relate directly to the unified row identifier, preventing split key redundancy.
3NF	Must be in 2NF; removes transitive dependencies (non-keys depend ONLY on the PK).	Attributes such as `class_name` are isolated in the `CLASSES` table rather than being stored in `STUDENTS`. This ensures that student records do not depend on class attributes transitively through a `class_id`.

Normalization Verification Details

1. **First Normal Form (1NF):** All table cells hold atomic values, and rows are uniquely identifiable via Primary Keys. This prevents the need for complex string parsing or multi-value cell management.
2. **Second Normal Form (2NF):** The design ensures that every non-key attribute is fully functionally dependent on the entire Primary Key. This is particularly relevant in junction tables where the relationship itself is the entity.
3. **Third Normal Form (3NF):** By removing transitive dependencies, the system eliminates update anomalies. For example, changing a class name in the `CLASSES` table automatically reflects for all students linked via `class_id` without requiring multiple row updates in the `STUDENTS` table.

Task 3 (d): Data Dictionary Schema Layout

The following data dictionary serves as the master engineering reference, detailing data types, constraints, and integrity behaviors for the primary system tables.

1. Entity: STUDENTS

Purpose: Tracks system enrollment data for active student records.

Attribute Name	Data Type	Constraint / Key	Description / Default Behavior
`student_id`	INT	PRIMARY KEY (AI)	Unique auto-incrementing identifier.
`first_name`	VARCHAR(50)	NOT NULL	Legal given name.
`last_name`	VARCHAR(50)	NOT NULL	Family/surname prefix.
`date_of_birth`	DATE	NOT NULL	Valid date record for age screening.
`gender`	CHAR(1)	CHECK (M, F, O)	Single character code indicator.
`address`	TEXT	NULLABLE	Physical residential location details.
`enroll_status`	ENUM	NOT NULL	Enforced states (e.g., ‘Active’, ‘Suspended’).
`class_id`	INT	FOREIGN KEY	Links to `CLASSES`; ON UPDATE CASCADE.
`hostel_id`	INT	FOREIGN KEY	Links to `HOSTELS`; SET NULL on delete.
`accom_paid`	BOOLEAN	DEFAULT FALSE	Financial housing status clearance flag.
`emerg_contact`	VARCHAR(100)	NOT NULL	Emergency supervisor contact string.

2. Entity: LECTURERS

Purpose: Organizes institutional academic staff details.

Attribute Name	Data Type	Constraint / Key	Description / Default Behavior
`lecturer_id`	INT	PRIMARY KEY (AI)	Unique instructor lookup id.
`name`	VARCHAR(100)	NOT NULL	Instructor corporate full name.
`specialty`	VARCHAR(100)	NULLABLE	Primary subject academic domain competency.
`managerial_role`	VARCHAR(50)	NULLABLE	System administrative assignments.
`allocated_hours`	INT	DEFAULT 0	Tracked weekly academic loading hours.
`department_id`	INT	FOREIGN KEY	Links to `DEPARTMENTS`; ON DELETE RESTRICT.

3. Entity: ASSESSMENTS

Purpose: Records performance criteria metrics and continuous assessments.

Attribute Name	Data Type	Constraint / Key	Description / Default Behavior
`assessment_id`	INT	PRIMARY KEY (AI)	Unique evaluation entry record sequence.
`student_id`	INT	FOREIGN KEY	Target student; ON DELETE CASCADE.
`subject_code`	VARCHAR(20)	FOREIGN KEY	Evaluated unit code; ON UPDATE CASCADE.
`term`	VARCHAR(10)	NOT NULL	Active school semester interval block.
`year`	INT	NOT NULL	Calendar year parameter.
`cat_1_score`	DECIMAL(5,2)	DEFAULT 0.00	Continuous Assessment Test 1 score.
`cat_2_score`	DECIMAL(5,2)	DEFAULT 0.00	Continuous Assessment Test 2 score.
`exam_score`	DECIMAL(5,2)	DEFAULT 0.00	Final sitting examination point result.
`attendance_pct`	DECIMAL(5,2)	DEFAULT 100.00	Tracked live lecture attendance percent log.

Conclusion

The structural design presented in this report ensures a robust and scalable database foundation. By adhering to 3NF standards and clearly defining attribute classifications and data dictionaries, the system is well-equipped to maintain data integrity and support complex academic workflows efficiently.